

Steven Zhang

(740) 278-5741 | zhang.st3ven1@gmail.com | linkedin.com/in/steven-zhang8 | [Portfolio](#) | Columbus, OH

EDUCATION

Ohio State University, College of Engineering

Columbus, OH

B.S. in Computer Science and Engineering | Minor: Mathematics | GPA: 3.7

TECHNICAL SKILLS

Languages: Python, Java, JavaScript (ES6+), SQL, C, Ruby, Assembly, MATLAB

Frameworks & Libraries: Flask, FastAPI, Spring Boot, React 18, Angular, Pydantic, SQLAlchemy, Chart.js, react-chartjs-2, Vite, Supabase, TensorFlow

Databases & Caching: PostgreSQL, MySQL, Redis

Tools & Platforms: Git/GitHub, Bitbucket, Docker, Docker Compose, Linux, AWS, Wireshark, Splunk, Metasploit, Nmap, VS Code, Eclipse, DataGrip

AI-Assisted Development: Claude Code, Codex

Other: Microsoft Excel, Canva | Fluent in Mandarin

PROJECTS

FITT — AI-Powered Fitness & Nutrition Tracker

github.com/jmulyadi/FITT

Backend Contributor | 6-Person Team (Python, FastAPI, Supabase/PostgreSQL, Groq AI, Docker)

- Developed FastAPI REST endpoints for workout logging, nutrition tracking, and recovery metrics, serving structured JSON to the React frontend
- Integrated Groq AI LLM API to power a conversational coaching feature generating personalized training and nutrition recommendations from user data
- Connected Supabase PostgreSQL with JWT-authenticated, row-level-secured queries for user-scoped fitness data, exercise history, and daily nutrition logs
- Wired ExerciseDB and Open Food Facts APIs into backend routes to enrich data with exercise metadata and macronutrient breakdowns
- Designed the REST API contract between frontend and backend teams, defining request/response schemas and error-handling conventions
- Implemented Pydantic schema validation on all request bodies to enforce type safety and auto-generate OpenAPI documentation for frontend integration
- Containerized the FastAPI service with Docker for reproducible local development across the team
- Collaborated within the 6-person team to integrate backend and frontend, aligning on shared API schemas and resolving contract mismatches through iterative review
- Set up Docker-based local environments so the team could run the backend identically across machines

Retirement Account Dashboard — Full-Stack Web Application

github.com/BigMuscleSmoe/retirement-dashboard

Personal Project (Java 17, Spring Boot 3, Spring Security, Angular, PostgreSQL, Docker, GitHub Actions)

- Built a Spring Boot REST API with 10 endpoints for account management, contribution tracking, balance history, and retirement projections
- Used Claude Code and Codex to scaffold JPA entities and controller boilerplate and to weigh auth-design options, then **reviewed, validated, and refined** all AI-generated code for correctness and security before integrating; applied AI-assisted debugging during development
- Implemented stateless JWT auth with access/refresh token flow, BCrypt hashing, and role-based access control via the Spring Security filter chain
- Wrote Flyway SQL migrations for schema evolution; structured JPA entities with relational mappings and Jakarta Bean Validation constraints
- Containerized the full stack with Docker Compose; set up GitHub Actions CI with automated build, test, and SAST scans; applied OWASP security headers
- Wrote integration tests with MockMvc and Testcontainers to validate endpoint behavior, auth flows, and database interactions
- Implemented global exception handling with @ControllerAdvice returning structured JSON error responses with appropriate HTTP status codes
- Built the Angular frontend against the REST API, with auth-guarded routes and reactive forms for accounts, contributions, and projection views
- Implemented the retirement-projection logic that models contribution growth and balance over time, served through dedicated REST endpoints

URL Shortener with Analytics — Backend Service

github.com/BigMuscleSmoe/url-shortener

Personal Project (Python, Flask, PostgreSQL, Redis, SQLAlchemy, Alembic, pytest, Docker)

- Built a Flask REST API with three endpoints: short-link creation with URL validation, 302 redirect with async click recording, and per-link analytics with clicks-by-day and top-referrer aggregations
- Used Claude Code and Codex to scaffold Flask route boilerplate and surface design options for short-code generation, then **reviewed, validated, and refined** the generated code before integrating; applied AI-assisted debugging to diagnose and resolve defects
- Implemented base62 random code generation with unique-constraint collision retry; documented tradeoffs vs. hash-based and sequential approaches
- Added Redis cache-aside on the redirect path with TTL-based expiration and negative caching for invalid codes, reducing database lookups for high-traffic links to a single cache-warming query
- Applied per-IP rate limiting on the create endpoint via Flask-Limiter backed by Redis for consistency across multiple app instances; returns 429 on limit breach
- Indexed links.code (unique) and clicks(link_id, clicked_at) to keep redirect lookups O(log n) and stats queries index-only
- Decoupled analytics writes from the redirect path using a background thread, so click recording never blocks the 302 response
- Wrote pytest integration tests covering create, redirect, 404 handling, rate limiting, and stats aggregation
- Versioned the PostgreSQL schema with Alembic migrations for repeatable, reviewable database changes

EXPERIENCE

Ohio State University

Columbus, OH

Teaching Assistant / Grader — Computer Science

Jan 2025 — May 2026

- Support 100+ students with debugging, Java syntax, and OOP concepts through office hours, discussion boards, and email
- Grade programming assignments and projects with consistent, timely feedback; coordinate standards with fellow TAs
- Lead weekly office hours and recitations, explaining recursion, data structures, and design patterns to students at varying skill levels
- Mentor students one-on-one on project architecture decisions, code organization, and version-control best practices
- Prepare worked code examples and debugging walkthroughs to reinforce core concepts during recitations and office hours

CERTIFICATIONS & INVOLVEMENT

- Google Cybersecurity Professional Certificate — SIEM tools, incident response, and vulnerability management
- CompTIA Security+ — In Progress — Studying network security, risk management, and cryptography
- Cyber Security Club, Ohio State (2024–2026) — CTF and pentesting with Wireshark, Metasploit, and Nmap